Facial Expression Recognition System using Convolutional Neural Networks

Shayan Ali
Department of Computer Science & Deep Learning

July 3, 2026

Abstract

This documentation details the implementation of a lightweight Convolutional Neural Network (CNN) engineered to classify facial expressions into distinct emotional categories. The model processes grayscale images utilizing mini-batch configurations to capture structural spatial hierarchies while preventing excessive computational overhead during the training phase.

Contents

1 Introduction **2**

2 Data Pipeline and Preprocessing **2**
2.1 Image Normalization and Scaling 2
2.2 Understanding Batch Sizes 2

3 Model Architecture **2**
3.1 Architectural Breakdown 3

4 Training Optimization and Loss Trends **3**
4.1 Non-Linearity in Validation Metrics 3

5 Evaluation Metrics **3**
5.1 The Confusion Matrix 3
5.2 Summary Metrics 4

1 Introduction

Facial Expression Recognition (FER) is a pivotal task in computer vision and human-computer interaction. This report outlines the technical documentation for constructing, training, and assessing a deep learning system trained on pixelated facial data to recognize seven distinct expressions.

2 Data Pipeline and Preprocessing

Efficient data handling is paramount to ensuring continuous data streams to the processing unit without bottlenecking hardware resource limitations.

2.1 Image Normalization and Scaling

Input pixel dimensions are standardized to a matrix footprint of 48×48 pixels over a single grayscale channel. Pixel values are mathematically normalized to scale feature constraints between an inclusive bounds of $[0, 1]$ using a min-max scaling rule:

$$x_{\text{norm}} = \frac{x}{255.0} \tag{1}$$

2.2 Understanding Batch Sizes

The pipeline uses a batch configuration of `BATCH_SIZE = 128`. It is critical to differentiate between internal layer filter multiplication and batch handling:

- **Image Processing:** Individual image matrices are independently multiplied via sliding element-wise kernels inside the convolutional operations regardless of the overall batch parameter.
- **Batch Update Mechanism:** The batch size governs the global number of forward-pass sequences evaluated in parallel execution before computing the mean group loss. Gradient adjustments via optimization algorithms are executed exactly once per full batch sweep to guarantee stable backpropagation trajectories.

3 Model Architecture

The architectural pipeline follows a sequential feed-forward paradigm optimized for high iteration execution speed and high abstraction retrieval.

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(48, 48, 1)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(7, activation='softmax')
])
```

Listing 1: TensorFlow Implementation of the CNN Model

3.1 Architectural Breakdown

1. **Convolutional Block 1:** Consists of 32 kernels (3×3) mapping early architectural low-level edges. Followed by a 2×2 downsampling `MaxPooling2D` layer.
2. **Convolutional Block 2:** Deploys 64 feature kernels (3×3) mapping abstract expression configurations. Followed by a 2×2 downsampling sequence.
3. **Dense Classifiers:** Vector values are flattened into a single feature stream linked to a fully-connected layer containing 128 nodes with a 50% dropout factor (`Dropout(0.5)`) to prevent structural overfitting.
4. **Output Layer:** Employs a multi-class Categorical Softmax function across 7 discrete emotional bins.

4 Training Optimization and Loss Trends

The network undergoes iterative gradient updates utilizing the Adam optimizer over an explicit execution ceiling of 10 structural training epochs.

4.1 Non-Linearity in Validation Metrics

While the training dataset loss typically presents a smooth, monotonic decline curve, the validation metric paths remain highly non-linear, presenting jagged characteristics. This normal behavior stems from:

1. **Unseen Parameters:** The validation pool is strictly excluded from backpropagation update loops. Minor tweaks that positively lower the internal loss of an isolated training batch may temporarily cause general misclassifications over the validation data.
2. **Mini-Batch Sampling Differences:** Training steps aggregate and smooth loss scores across multiple mini-batches every epoch, whereas validation steps record isolated static metrics at the absolute terminus of the epoch loop.

5 Evaluation Metrics

To evaluate true system robustness beyond raw accuracy figures, comprehensive diagnostic pipelines are established using continuous testing verification frameworks.

5.1 The Confusion Matrix

Performance imbalances across structurally resembling emotional groups (e.g., *Sad* vs. *Neutral*) are visualized via a 7×7 Confusion Matrix grid.

- **The True Diagonal Axis:** Tracks identical alignments where the network's predicted classification label directly matches the ground truth tag. Maximizing pixel values along this specific axis is the target configuration goal.
- **Off-Diagonal Noise:** Any coordinate index drifting outside the primary diagonal axis explicitly highlights misclassifications, pointing out feature extraction gaps in the model.

5.2 Summary Metrics

The categorical cross-entropy loss function is computed alongside total sample accuracy over the final test database:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Evaluation Samples}} \tag{2}$$